

Examen PF 2026

30.01.2026

Varianta 1.3

1. Ce tip are funcția de mai jos?

```
\f g x -> (f x, g x)
```

- a) $(a \rightarrow b) \rightarrow (a \rightarrow c) \rightarrow a \rightarrow (b, c)$
- b) Nicio variantă
- c) $(a \rightarrow b) \rightarrow (c \rightarrow d) \rightarrow (a, c) \rightarrow (b, d)$
- d) $(a \rightarrow b) \rightarrow (b \rightarrow a) \rightarrow a \rightarrow (b, a)$

2. Ce se obține după instrucțiunea de mai jos?

```
filter (== "5") [1,2,3,4,5]
```

- a) [5]
- b) Nicio variantă
- c) 1
- d) Eroare

3. Ce valoare are x în codul de mai jos?

```
x = let
    x = 1
    y = x
in
    x + y
```

- a) Eroare
- b) Nicio variantă
- c) 1
- d) 2

4. Care dintre afirmațiile de mai jos este adevărată în Haskell?

- a) O instrucțiune if necesită mereu și then și else
- b) Nicio variantă
- c) Este posibil să reatribui o valoare unei variabile
- d) Nu este posibil să refolosești numele unei variabile

5. Ce tip are construcția de mai jos?

```
\x -> case x of (True, "Foo") -> show True ++ "Foo"
```

- a) $(\text{Bool}, \text{String}) \rightarrow \text{String}$
- b) Nicio variantă
- c) $\text{Either Bool String} \rightarrow \text{String}$
- d) $\text{Show a} \Rightarrow (\text{Bool}, \text{String}) \rightarrow a$

6. Ce tip are funcția de mai jos?

```
justBoth a b = [Right a, Left b]
```

- a) $a \rightarrow b \rightarrow [\text{Either a}, \text{Either b}]$
- b) $a \rightarrow b \rightarrow [\text{Maybe a}]$
- c) $a \rightarrow b \rightarrow [\text{Either b a}]$
- d) $a \rightarrow b \rightarrow [\text{Either a b}]$

7. De ce expresia "3" ++ "FOO" nu este validă în Haskell?
- a) Pentru că "FOO" nu este o valoare numerică, ci un șir de caractere
 - b) Pentru că operatorul ++ este predefinit doar pentru tipul Int
 - c) Expresia este validă în Haskell
 - d) Nicio variantă
8. Ce se obține după instrucțiunea de mai jos?
- ```
filter (\x -> (2 < x) && (x < 6)) [1..10]
```
- a) [3,4,5]
  - b) Eroare
  - c) [2,3,4,5,6]
  - d) Nicio variantă
9. Ce expresie este echivalentă cu instrucțiunea de mai jos?
- ```
do x <- z
  g y
  return (f x)
```
- a) `z >> \x -> g y >>= return (f x)`
 - b) Nicio variantă
 - c) `z >>= \x -> g y >> return (f x)`
 - d) `z >> \x -> g y >> return (f x)`
10. Ce face funcția de mai jos?
- ```
foldr (\x xs -> xs ++ [x]) []
```
- a) Întoarce elementele unei liste
  - b) Nu modifică lista primită ca parametru
  - c) Eroare
  - d) Nicio variantă
11. Ce se obține după instrucțiunea de mai jos?
- ```
(++) <$> ["1", "2"] <*> ["3", "4"]
```
- a) [4,5,5,6]
 - b) Eroare
 - c) Nicio variantă
 - d) ["13", "14", "23", "24"]
12. Ce tip are construcția de mai jos?
- ```
\x xs -> return x : xs
```
- a) `a -> [a] -> Monad [a]`
  - b) `Monad m => a -> [m a] -> [m a]`
  - c) Nicio variantă
  - d) `Monad m => a -> [m a] -> [m a]` (Nota: Pare duplicat sau ușor diferit în imagine, dar b este varianta standard corectă)

13. Presupunând că avem acces la `Data.List`, care dintre instrucțiunile de mai jos este corectă în Haskell?

- a) `head . take 5 $ [1..17]`
- b) `head . take 5 [1..17]`
- c) `head . take $ 5 [1..17]`
- d) `head . take $ [1..17]`

14. Care cod este o instanță a clasei `Functor` pentru tipul de mai jos (astfel încât să satisfacă și legile din definiția unui functor)?

```
data Container x = Things x [x]
```

```
instance Functor Container where
 ????
```

- a) `fmap f (Things x ys) = f (Things x ys)`
- b) `fmap f (Things x ys) = Things (f x) (map f ys)`
- c) `fmap f (Things x ys) = Things (f x) [f x]`
- d) `fmap f (Things x ys) = Things (f x) (map f ys)`

---

22. Ce se obține după instrucțiunea de mai jos?

```
[1 ..] !! 5
```

- a) Nicio variantă
- b) 5
- c) 6
- d) Eroare

23. Ce tip are funcția de mai jos?

```
foo x = do putStrLn x
 y <- getLine
 return (length y)
```

- a) `String -> IO String`
- b) `String -> IO Int`
- c) `IO Int`
- d) `IO String -> IO Int`

24. Ce se obține după instrucțiunea de mai jos?

```
map (++ 3) [1,2,3]
```

- a) `[1,2,3]`
- b) `[4,5,6]`
- c) Eroare
- d) Nicio variantă

25. Ce se obține după instrucțiunea de mai jos?

```
foldr (\n b -> n == 3 && b) True [3,1,3]
```

- a) `False`
- b) Eroare
- c) Nicio variantă
- d) `True`

26. Ce valoare are `h 5` în codul de mai jos?

```
h x = g z
 where
 g z = z ^ 2
 z = 2
```

- a) 4
- b) Nicio variantă
- c) 25
- d) Eroare

27. Dacă `f :: a -> b`, atunci ce tip are `map (.f)`?

- a) `[a] -> [b]`
- b) `[b -> c] -> [a -> c]`
- c) `[c -> a] -> [c -> b]`
- d) `(b -> c) -> [a -> c]`

28. Care dintre funcțiile de mai jos înmulțește primul argument la al doilea?

- a) `f x = \y -> x * y`
- b) `f x = \y . x * y`
- c) `f = \ x y -> x * x`
- d) `f x x = x * x`